
NOTES BY Sonu ILYas MughaL

Introduction to Programing (CS201)**Programing:**

A programming language is a set of commands, instructions, and other syntax use to create a software program. Languages that programmers use to write code are called "high-level languages." This code can be compiled into a "low-level language," which is recognized directly by the computer hardware.

Syntax:

Every spoken language has a general set of rules for how words and sentences should be structured. These rules are collectively known as the language syntax. In computer programming, syntax serves the same purpose, defining how declarations, functions, commands, and other statements should be arranged.

Program:

Program is a common computer term that can be used as both a noun and a verb. A program (noun) is executable software that runs on a computer. It is similar to a script, but is often much larger in size and does not require a scripting engine to run. Instead, a program consists of compiled code that can run directly from the computer's operating system.

What is C++?

C++ is a cross-platform language that can be used to create high-performance applications.

C++ was developed by Bjarne Stroustrup at Bell labs in 1979, as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

The language was updated 3 major times in 2011, 2014, and 2017 to C++11, C++14, and C++17.

Why Use C++

C++ is one of the world's most popular programming languages.

C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object-oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

As C++ is close to C# and Java, it makes it easy for programmers to switch to C++ or vice versa

First program:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    return 0;
}
```

Let's break up the following code to understand it better:

Example explained

Line 1: `#include <iostream>` is a **header file library** that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.

Line 2: `using namespace std` means that we can use names for objects and variables from the standard library.

Line 3: A blank line. C++ ignores white space.

Line 4: Another thing that always appear in a C++ program, is `int main()`. This is called a **function**. Any code inside its curly brackets `{ }` will be executed.

Line 5: `cout` (pronounced "see-out") is an **object** used to output/print text. In our example it will output "Hello World".

Note: Every C++ statement ends with a semicolon `;`.

Note: The body of `int main()` could also been written as:
`int main () { cout << "Hello World! "; return 0; }`

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 6: `return 0` ends the main function.

Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `using namespace std` line can be omitted and replaced with the `std` keyword, followed by the `::` operator for some objects:

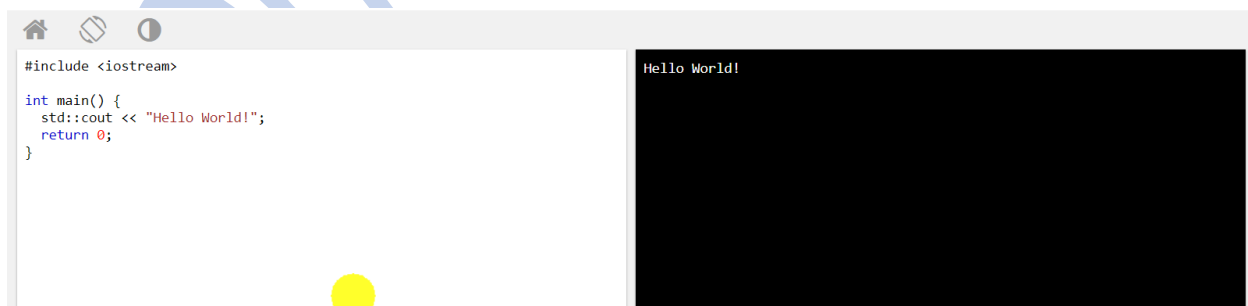
Example

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello World!";  
    return 0;  
}
```

It is up to you if you want to include the standard namespace library or not.

Run program



```
#include <iostream>  
  
int main() {  
    std::cout << "Hello World!";  
    return 0;  
}
```

```
Hello World!
```

C++ Output (Print Text)

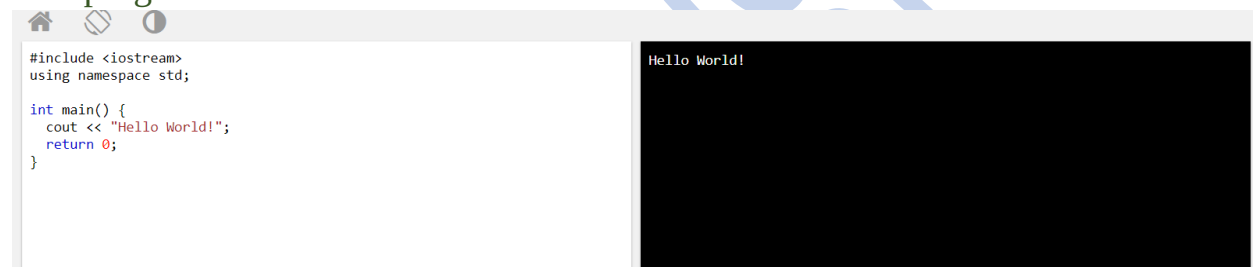
The `cout` object, together with the `<<` operator, is used to output values/print text:

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello World!";
    return 0;
}
```

You can add as many `cout` objects as you want. However, note that it does not insert a new line at the end of the output:

Run program:



```
#include <iostream>
using namespace std;

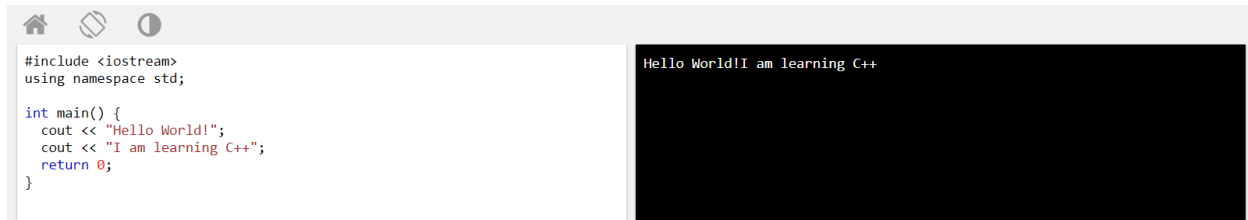
int main() {
    cout << "Hello World!";
    return 0;
}
```

```
Hello World!
```

Example

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello World!";
    cout << "I am learning C++";
    return 0;
}
```



```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    cout << "I am learning C++";
    return 0;
}
```

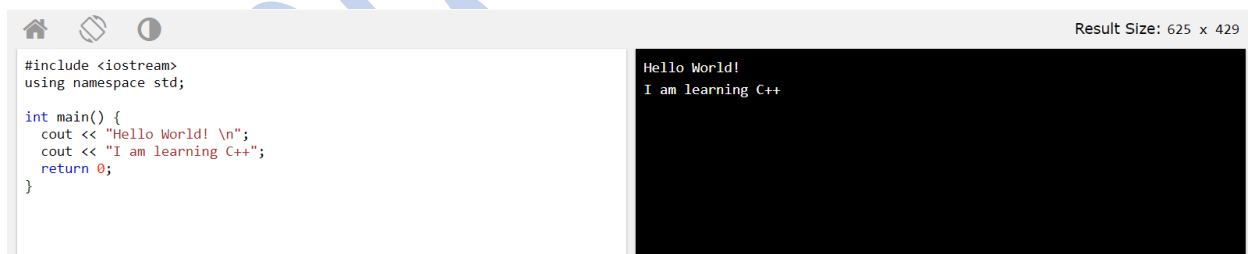
Hello World! I am learning C++

New Lines

To insert a new line, you can use the `\n` character:

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello World! \n";
    cout << "I am learning C++";
    return 0;
}
```



```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World! \n";
    cout << "I am learning C++";
    return 0;
}
```

Hello World!
I am learning C++

Result Size: 625 x 429

Tip: Two `\n` characters after each other will create a blank line:

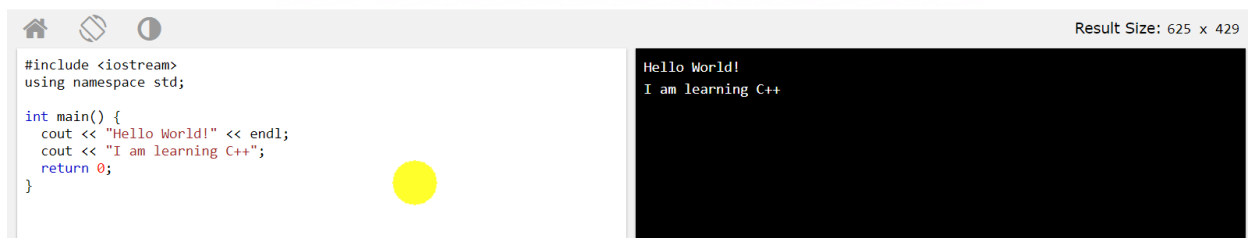
Another way to insert a new line, is with the `endl` manipulator:

Example

```
#include <iostream>
using namespace std;
```

```
int main() {  
    cout << "Hello World!" << endl;  
    cout << "I am learning C++";  
    return 0;  
}
```

Run program

A screenshot of a C++ IDE. The left pane shows the source code:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello World!" << endl;  
    cout << "I am learning C++";  
    return 0;  
}
```

 The right pane shows the output:

```
Hello World!  
I am learning C++
```

 The top right corner of the IDE window indicates "Result Size: 625 x 429".

Both `\n` and `endl` are used to break lines. However, `\n` is used more often and is the preferred way.

C++ Comments

Comments can be used to explain C++ code, and to make it more readable. It can also be used to prevent execution when testing alternative code. Comments can be single-lined or multi-lined.

Single-line comments start with two forward slashes (`//`).

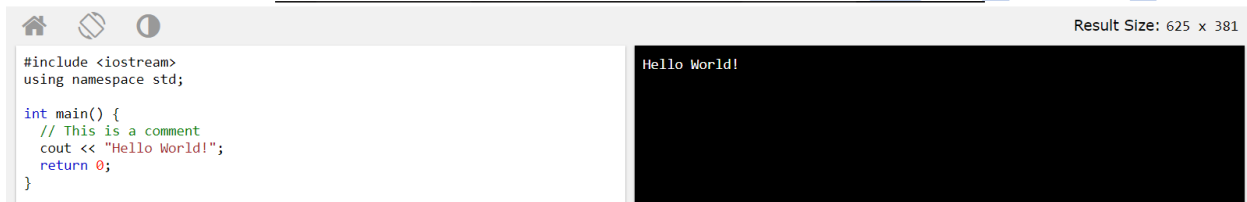
Any text between `//` and the end of the line is ignored by the compiler (will not be executed).

This example uses a single-line comment before a line of code:

Example

```
// This is a comment  
cout << "Hello World!";
```

Run program

A screenshot of a C++ IDE. On the left, a code editor shows the following code:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    // This is a comment  
    cout << "Hello World!";  
    return 0;  
}
```

On the right, a terminal window displays the output "Hello World!". The top right corner of the IDE window indicates "Result Size: 625 x 381".

This example uses a single-line comment at the end of a line of code:

Example

```
cout << "Hello World!"; // This is a comment
```

C++ Multi-line Comments

Multi-line comments start with `/*` and ends with `*/`.

Any text between `/*` and `*/` will be ignored by the compiler:

Example

```
/* The code below will print the words Hello World!  
to the screen, and it is amazing */  
cout << "Hello World!";
```

[Run example »](#)



```
#include <iostream>
using namespace std;

int main() {
    /* The code below will print the words Hello World!
    to the screen, and it is amazing */
    cout << "Hello World!";
    return 0;
}
```

Hello World!

Single or multi-line comments?

It is up to you which you want to use. Normally, we use `//` for short comments, and `/* */` for longer.

C++ Variables

Variables are containers for storing data values.

In C++, there are different **types** of variables (defined with different keywords), for example:

- `int` - stores integers (whole numbers), without decimals, such as 123 or -123
- `double` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `char` - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `bool` - stores values with two states: true or false

Declaring (Creating) Variables

To create a variable, you must specify the type and assign it a value:

Syntax

type variable = value;

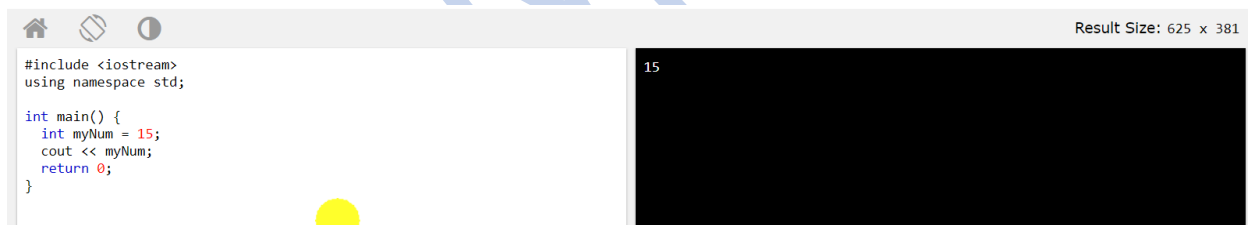
Where *type* is one of C++ types (such as `int`), and *variable* is the name of the variable (such as `x` or `myName`). The **equal sign** is used to assign values to the variable.

To create a variable that should store a number, look at the following example:

Example

Create a variable called `myNum` of type `int` and assign it the value `15`:

```
int myNum = 15;
cout << myNum;
```

A screenshot of a C++ IDE. The left pane shows the source code:

```
#include <iostream>
using namespace std;

int main() {
    int myNum = 15;
    cout << myNum;
    return 0;
}
```

 The right pane shows the output:

```
15
```

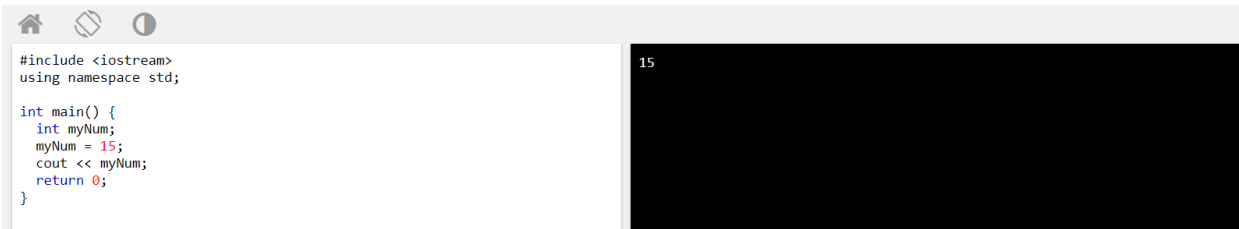
 The top right corner of the IDE window displays "Result Size: 625 x 381".

You can also declare a variable without assigning the value, and assign the value later:

Example

```
int myNum;
myNum = 15;
cout << myNum;
```

[Run example »](#)



```
#include <iostream>
using namespace std;

int main() {
    int myNum;
    myNum = 15;
    cout << myNum;
    return 0;
}
```

15

Note that if you assign a new value to an existing variable, it will overwrite the previous value:

Example

```
int myNum = 15; // myNum is 15
myNum = 10; // Now myNum is 10
cout << myNum; // Outputs 10
```



```
#include <iostream>
using namespace std;

int main() {
    int myNum = 15; // Now myNum is 15
    myNum = 10; // Now myNum is 10
    cout << myNum;
    return 0;
}
```

10

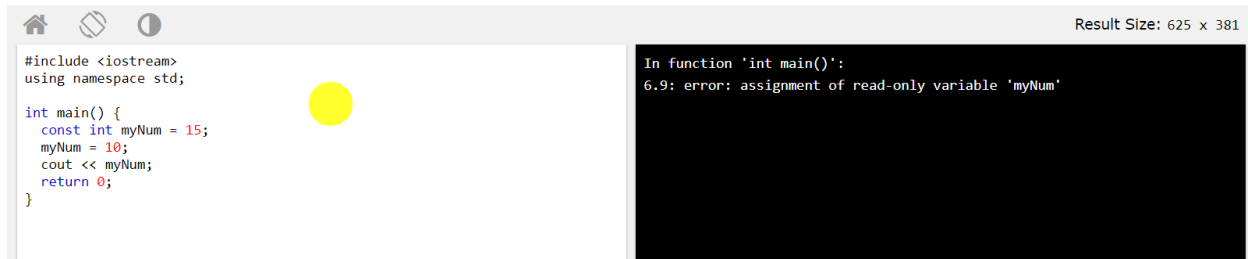
Result Size: 625 x 381

Constants

However, you can add the `const` keyword if you don't want others (or yourself) to override existing values (this will declare the variable as "constant", which means **unchangeable and read-only**):

Example

```
const int myNum = 15; // myNum will always be 15
myNum = 10; // error: assignment of read-only variable 'myNum'
```



```
#include <iostream>
using namespace std;

int main() {
    const int myNum = 15;
    myNum = 10;
    cout << myNum;
    return 0;
}
```

Result Size: 625 x 381

In function 'int main()':
6.9: error: assignment of read-only variable 'myNum'

Other Types

A demonstration of other data types:

Example

```
int myNum = 5;           // Integer (whole number without decimals)
double myFloatNum = 5.99; // Floating point number (with decimals)
char myLetter = 'D';     // Character
string myText = "Hello"; // String (text)
bool myBoolean = true;   // Boolean (true or false)
```

Display Variables

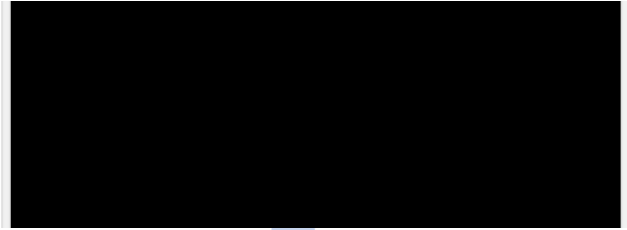
The `cout` object is used together with the `<<` operator to display variables.

To combine both text and a variable, separate them with the `<<` operator:

Example

```
int myAge = 35;
cout << "I am " << myAge << " years old.";
```

```
int main() {  
    int myAge = 35;  
    cout << "I am " << myAge << " years old."  
    return 0;  
}
```

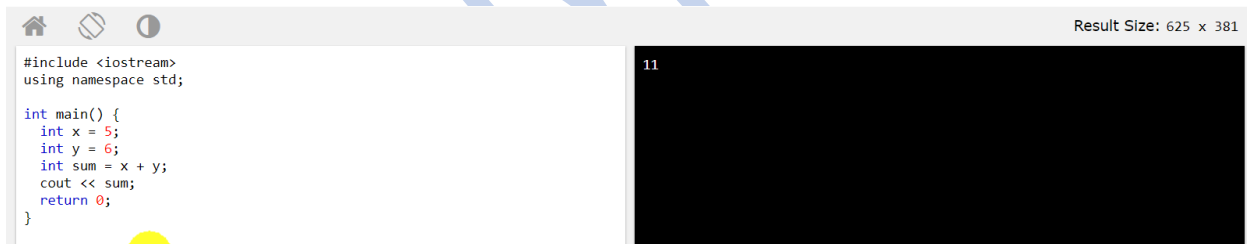


Add Variables Together

To add a variable to another variable, you can use the `+` operator:

Example

```
int x = 5;  
int y = 6;  
int sum = x + y;  
cout << sum;
```



Declare Many Variables

To declare more than one variable of the **same type**, you can use a comma-separated list:

Example

```
int x = 5, y = 6, z = 50;  
cout << x + y + z;
```

```
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 6, z = 50;
    cout << x + y + z;
    return 0;
}
```

61

C++ Identifiers

All C++ **variables** must be **identified** with **unique names**.

These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

Note: It is recommended to use descriptive names in order to create understandable and maintainable code.

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case sensitive (myVar and myvar are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as int) cannot be used as names